



Building Multimodal Agents With NVIDIA Nemotron and RAPIDS - Part 2

Antonio Rueda-Toicen

Senior Developer Relations Manager

Vaibhav Sabhahit

Senior Solutions Architect

Agenda

- Introduction to Bertopic and Concept
- Multimodal embeddings with CLIP and CRADIO-v4
- Dimensionality Reduction with Principal Component Analysis and UMAP
- Clustering with K-means and HDBSCAN
- cuML acceleration with RAPIDS

Notebook

- [Bertopic on Images](#) (Google Colab)

Setup

- [Configuring your NVIDIA API KEY to use models from build.nvidia.com](#)
- [Setting up GPUs on Kaggle Notebooks](#)
- [Setting up a HuggingFace token](#)
- [Setting Access to Kaggle Jupyter Server on VS Code](#)

Core question in topic modeling

"What is this data about?"

Subjective, for both humans and LLMs



Topic Modeling

🕒 45 min

Extract insights from massive text datasets using cuML's GPU-accelerated BERTopic

bertopic

data science

machine learning

nlp

cuml

Overview

Instructions

Basic idea

Topic modeling helps you discover hidden themes in large document collections—but traditional methods crawl when datasets grow to millions of records. This playbook shows how to process **40 million Amazon product reviews in minutes** using GPU-accelerated BERTopic.

BERTopic combines transformer embeddings with clustering to extract human-readable topics from text. By swapping CPU-based UMAP and HDBSCAN with GPU-accelerated versions from **RAPIDS cuML**, you get the same results dramatically faster—no code changes required.

- **Drop-in GPU acceleration:** Load `cuml.accel` and your existing UMAP/HDBSCAN code runs on GPU automatically
- **Scale to millions:** Process datasets that would take hours on CPU in minutes on GPU
- **Interactive visualizations:** Explore topic distributions, relationships, and document clusters

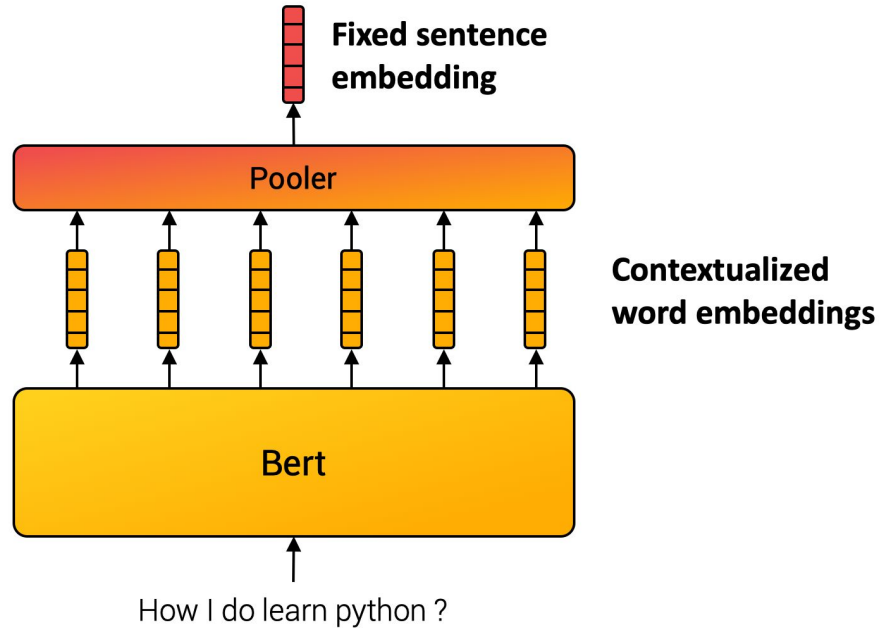
[Topic Modeling | DGX Station at build.nvidia.com](#)

Topic 1		Topic 2	
border	.4	space	.6
visa	.3	planet	.4
alien	.2	stars	.4
immigrant	.2	alien	.2

Document1: Topic 1: 1, Topic 2: 0
Document2: Topic 1: .1, Topic 2: .9
Document3: Topic 1: .85, Topic 2: .15
Document4: Topic 1: 0, Topic 2: 1

What is topic modeling?

Sentence Embeddings from BERT



[Embedding Atlas](#)

[BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#)

 AUGUST WESTER · UPDATED 2 MONTHS AGO

24

<> Code

Download

arXiv embeddings

Embeddings for over 250,000 ML papers on arXiv (and counting!)

 + 

Data Card

Code (0)

Discussion (0)

Suggestions (0)

About Dataset

These are the embeddings powering searchthe-arxiv.com, a semantic search engine for more than a decade's worth of ML papers published on arXiv. The embeddings are created by running OpenAI's `text-embedding-ada-002` model on an "augmented" abstract for each paper (title+authors+year+abstract). The papers are sourced from the [metadataset published by Cornell University](#) and filtered to include only papers belonging to at least one of the following categories:

`cs.cv`, `cs.lg`, `cs.cl`, `cs.ai`, `cs.ne`, and `cs.ro`

The dataset is updated on a weekly basis (in lockstep with the official arXiv metadataset).

If you find the dataset and/or searchthe-arxiv.com useful, consider giving the repo a ★ on [GitHub](#).

Usability ⓘ
8.13

License
[CC0: Public Domain](#)

Expected update frequency
Weekly

Tags
Computer Science

ML Arxiv Embeddings



SUMIT MISHRA · UPDATED 7 DAYS AGO



60



Code

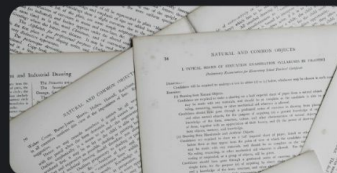


Download



arXiv Scientific Research Papers Dataset

A Curated Dataset of arXiv Papers with Titles, Summaries, Categories & Metadata.



Data Card

Code (21)

Discussion (0)

Suggestions (0)

About Dataset

Dataset Description: ArXiv Scientific Research Papers Dataset

This dataset is a **curated collection of research papers from arXiv**, covering various scientific fields such as **Artificial Intelligence, Machine Learning, computer science, mathematics and more**. It includes **titles, abstracts, categories, authors, publications and Updated dates** making it useful for various **machine learning and NLP tasks**. It can be used for several use cases given below

1. **Research Paper Classification** – Train a model to predict a paper's category based on its title or abstract.
2. **Recommendation Systems** – Suggest similar papers based on textual content or categories.
3. **Trend Analysis** – Analyze how research topics evolve.
4. **Text Summarization** – Develop models to generate concise summaries of scientific papers.
5. **Topic Modeling** – Identify hidden themes and emerging research areas.
6. **Author Impact Analysis** – Track contributions and collaborations across different domains.

Upvote if you like!! 🍌

Usability ⓘ

10.00

License

Apache 2.0

Expected update frequency

Quarterly

Tags

Education

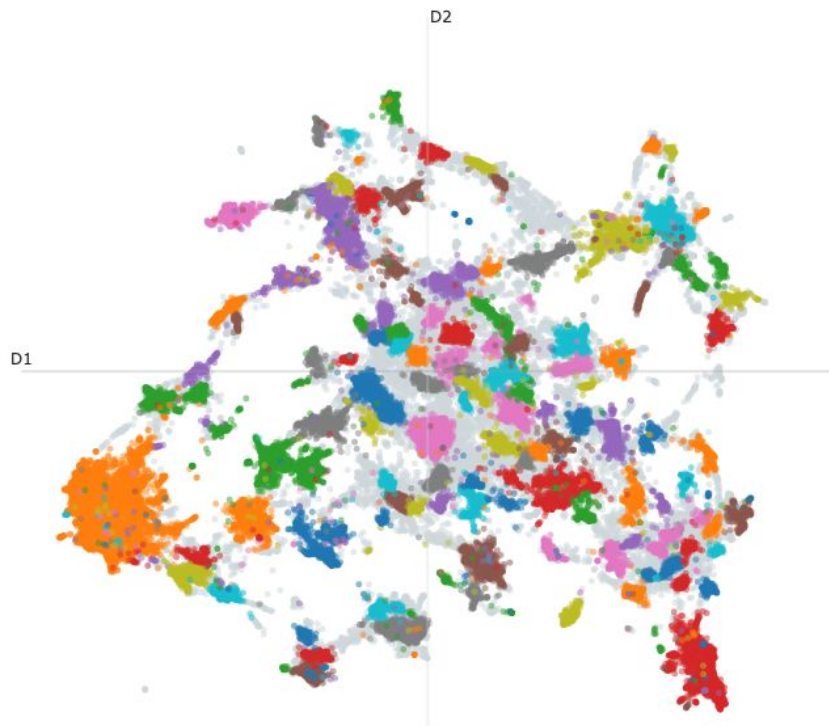
Artificial Intelligence

Research

Text Mining

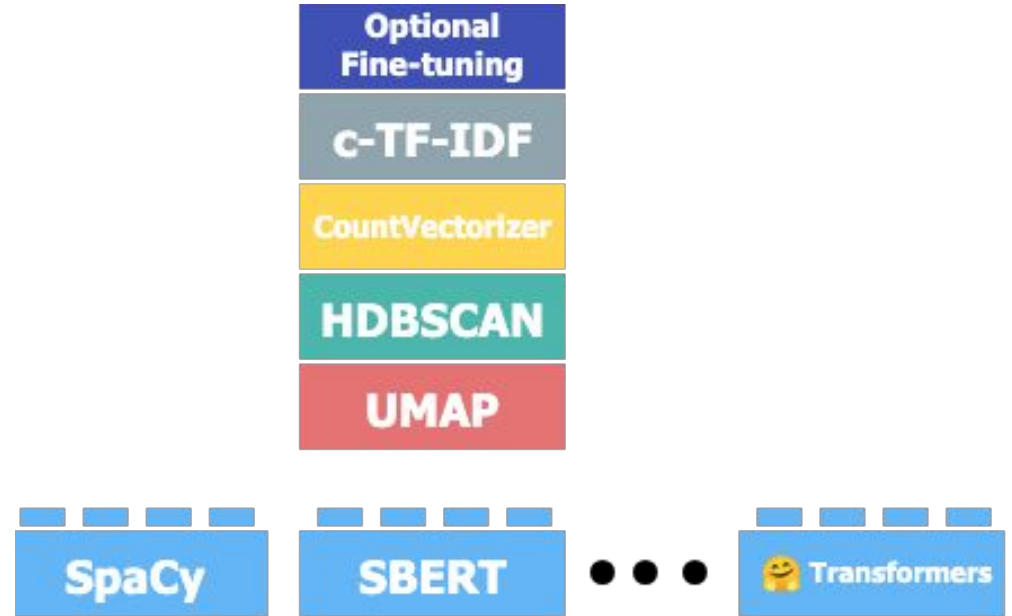
Transformers

Documents and Topics

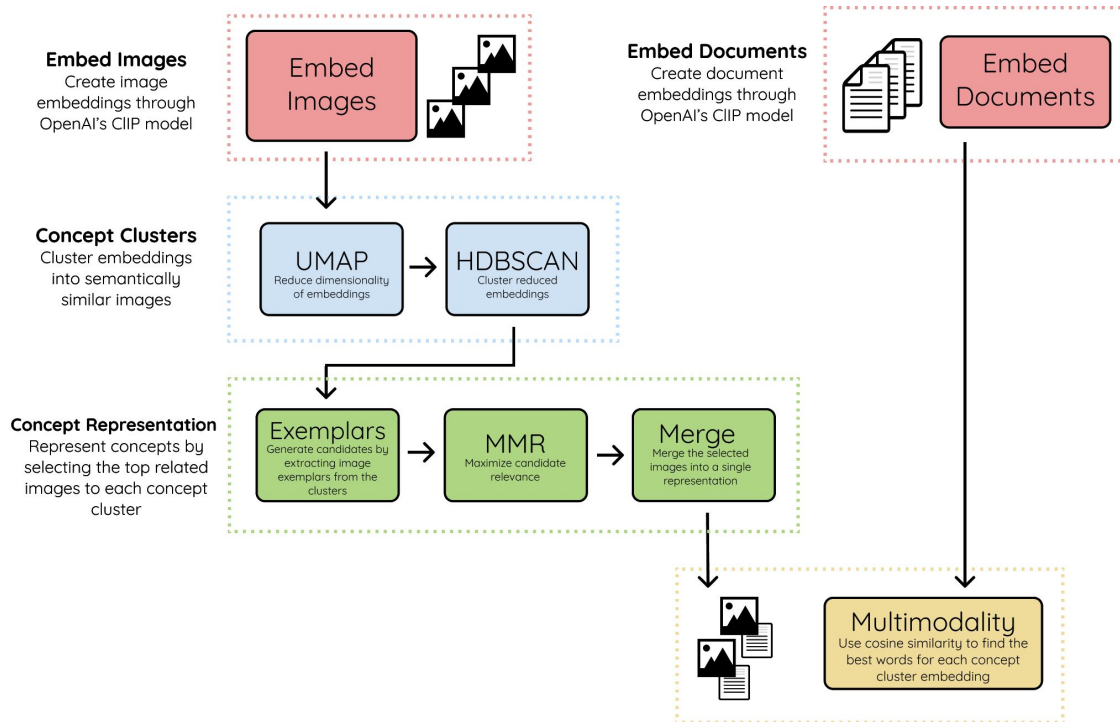
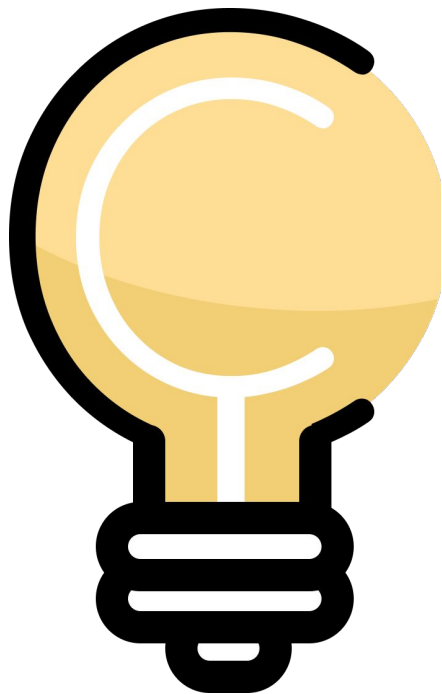


- Deep Reinforcement Learning Challenges
- Privacy-preserving federated learning
- Audio-Visual Speech Separation and Recognition
- Solving Partial Differential Equations using Neural Networks
- Adversarial Robustness in Deep Learning
- Deep Learning Theory
- Graph Embedding and Representation Learning
- Clinical Data Analysis and Missing Information Imputation
- Medical Image Segmentation for Cancer Diagnosis
- Optimization techniques for stochastic non-convex problems
- Wireless Communication Networks and Channel Estimation
- Multi-Armed Bandit Problem
- 3D Object Detection and Generation
- Generative Adversarial Networks (GANs) Training Challenges
- Recommender Systems and User Interactions
- Smart Grid Energy Management
- Causal Inference and Discovery
- Autonomous Vehicle Safety and Trajectory Planning
- Explainable Artificial Intelligence (XAI)
- Malware Detection using Machine Learning
- Fairness and Bias in Machine Learning
- Quantum Machine Learning
- Clustering Algorithms
- Time Series Forecasting
- Remote Sensing Image Classification
- Multi-Label Learning with Noisy Labels
- Distributed Machine Learning Optimization
- Few-shot Learning and Meta-Learning
- Traffic Prediction and Mobility in Urban Areas
- Anomaly Detection in Time Series using Deep Learning

Topic Modeling
with LLama 2



<https://maartengr.github.io/BERTopic/index.html>



<https://github.com/MaartenGr/Concept>



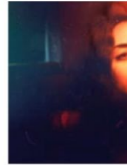
8.5M

photos



426k

photographers

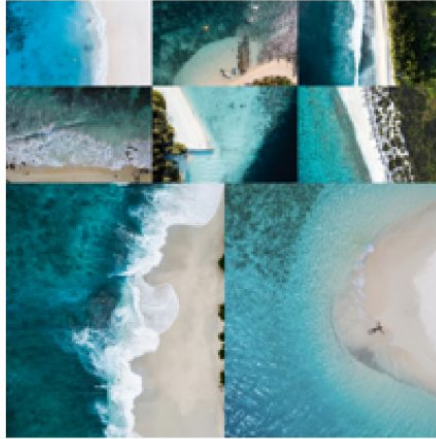


[Unsplash's image collection](#)

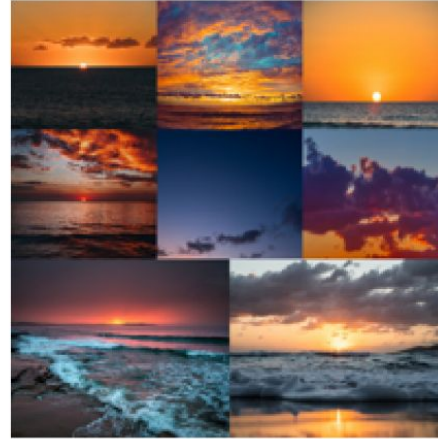
Concept 46:
hangfire, firewood, bonfire, fireside, fires



Concept 85:
ocean, tonga, island, islands, philippines



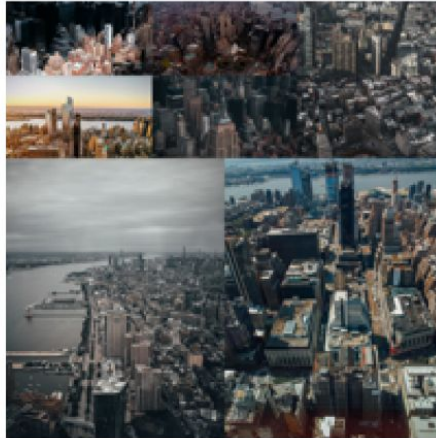
Concept 94:
sea, dawn, sunrise, sunset, subset



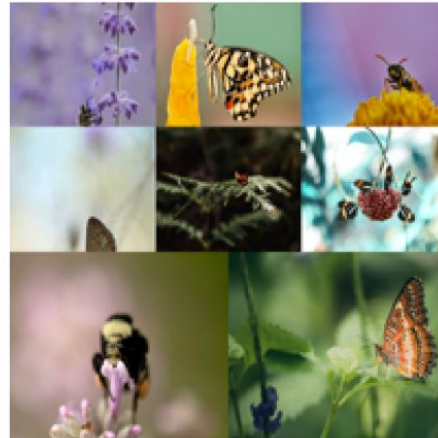
Concept 64:
wall, leaves, seasons, fall, garrison



Concept 74:
skyline, city, cities, nylanders, ny



Concept 9:
insect, formicinae, nectar, insects, bee



BERTopic: Neural topic modeling with a class-based TF-IDF procedure

[M Grootendorst](#) - arXiv preprint arXiv:2203.05794, 2022 - arxiv.org

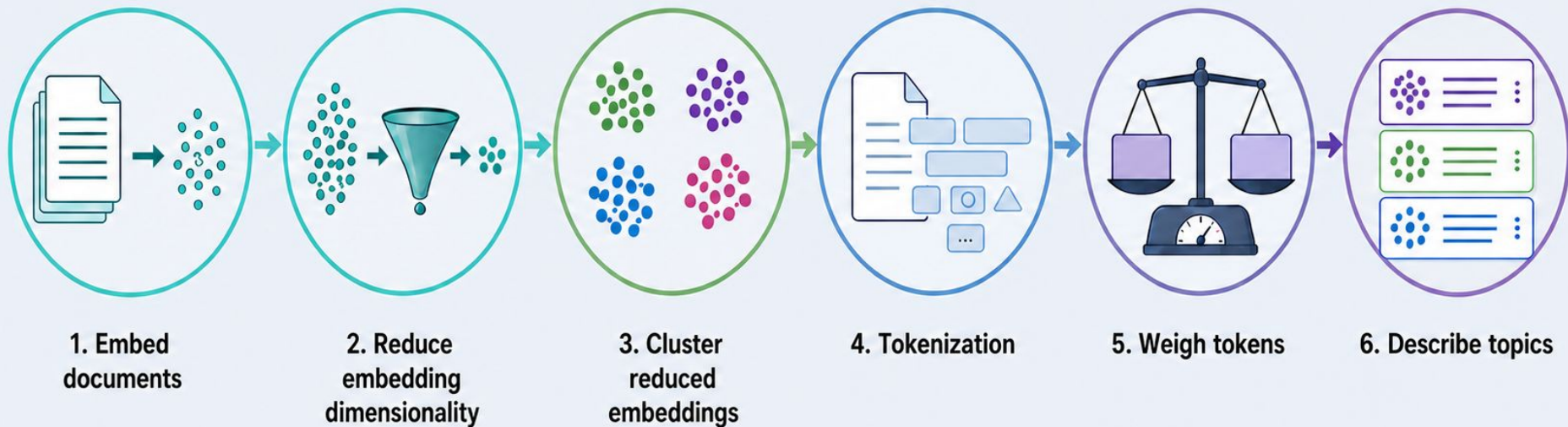
... We present **BERTopic**, a topic model that extends this process ... More specifically, **BERTopic** generates document embedding ... **BERTopic** generates coherent topics and remains compet...

☆ Save  Cite Cited by 5482 Related articles All 5 versions View as HTML 

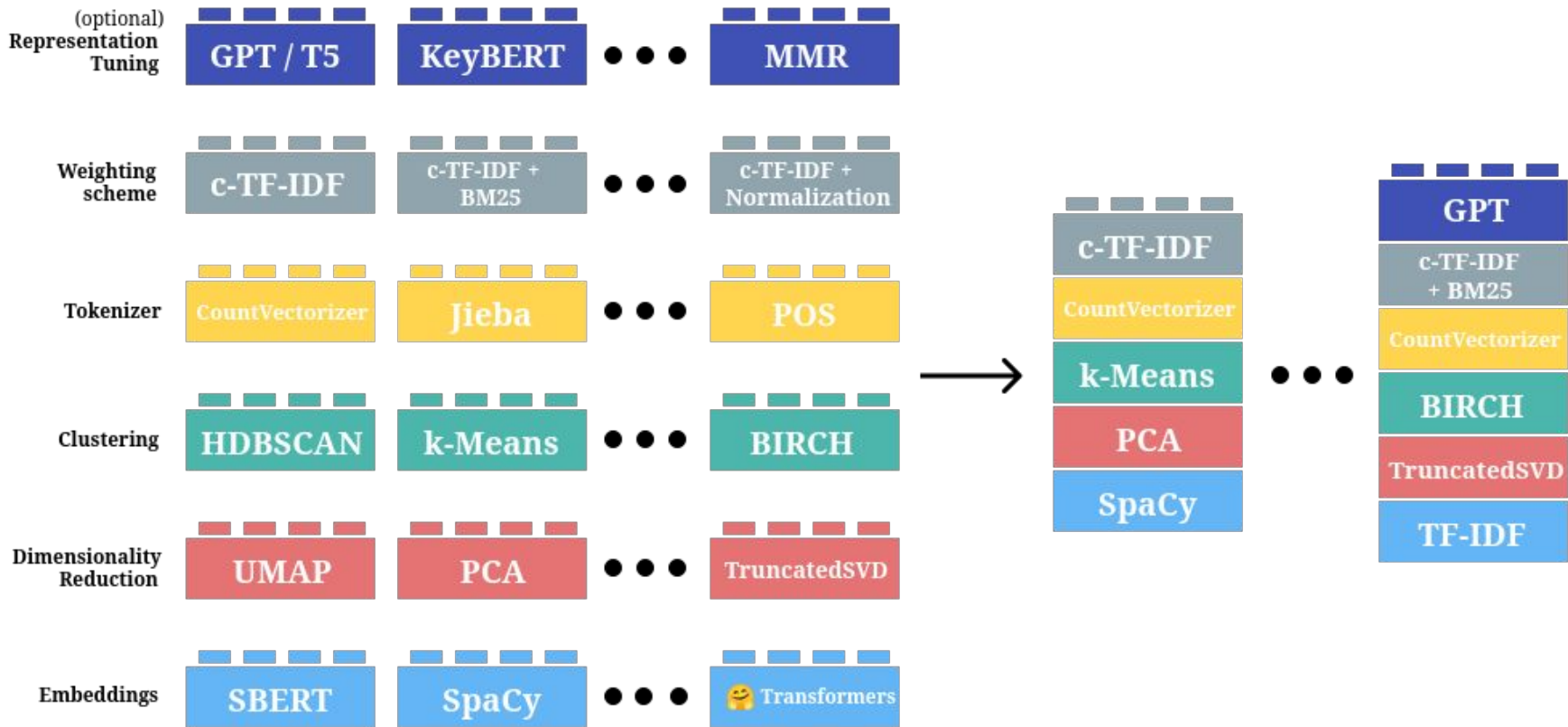


[ArXiv Link](#)

Steps for Bertopic



Exploratory Swapping of Components



c-TF-IDF

For a term x within class c :

$$W_{x,c} = \| \text{tf}_{x,c} \| \times \log \left(1 + \frac{A}{f_x} \right)$$

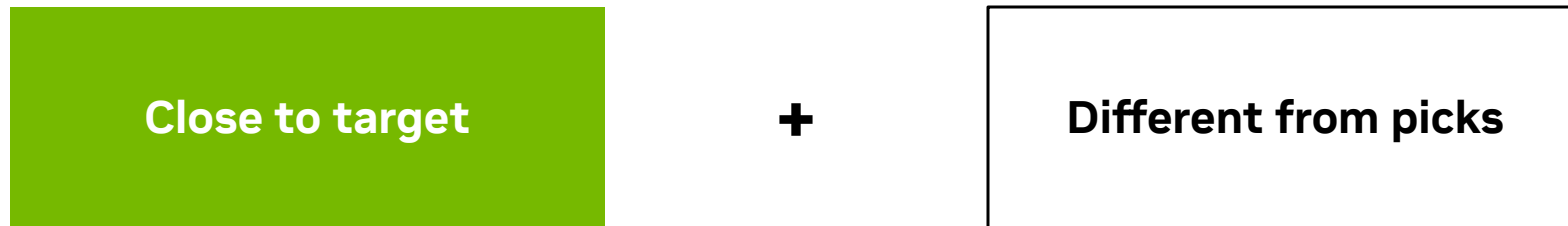
$\text{tf}_{x,c}$ = frequency of word x in class c

f_x = frequency of word x across all classes

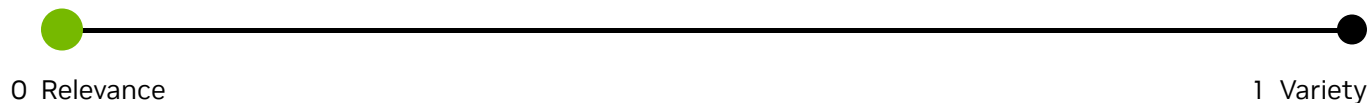
A = average number of words per class

Keyword extraction in BERTopic

MMR: Choose what fits, skip what repeats



$$\text{score} = \text{relevance} - \text{redundancy}$$

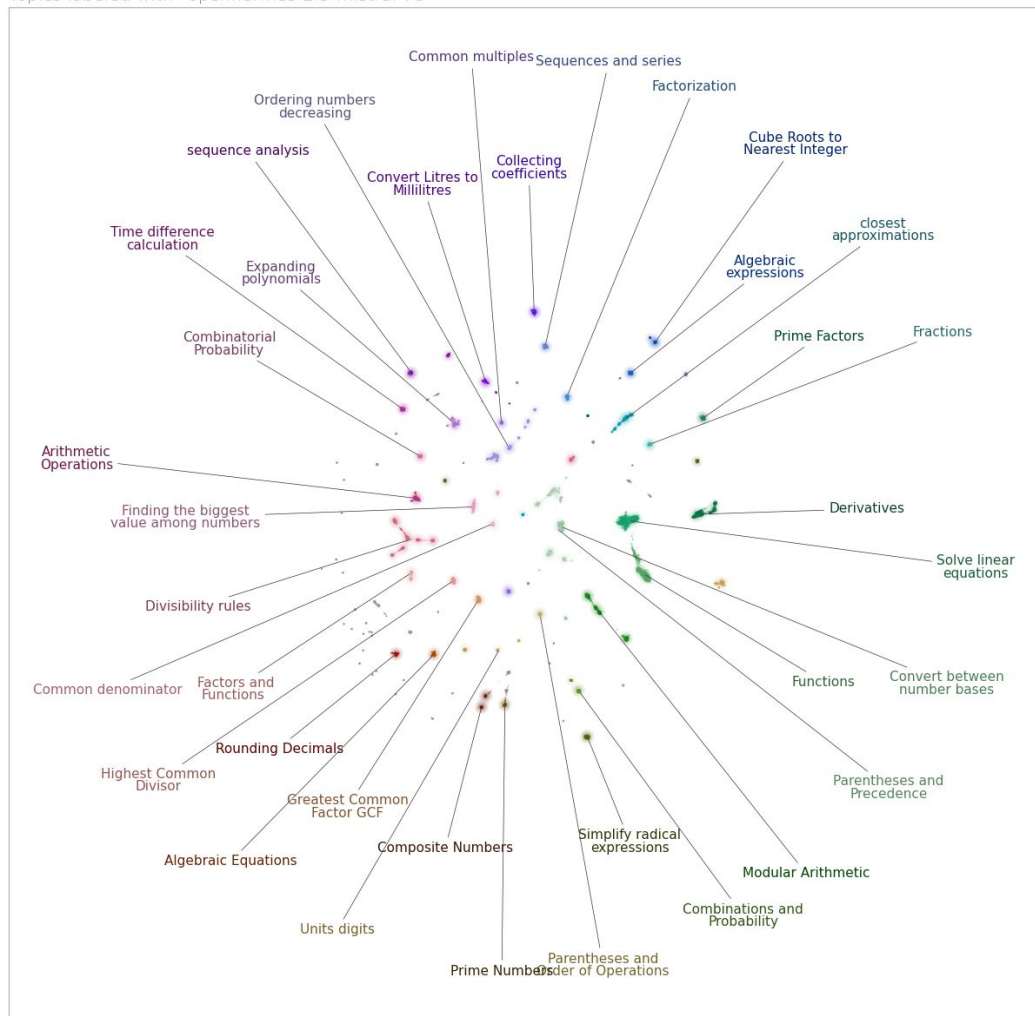


The Three Pillars of BERTopic

Topic Modeling + Exploration + Visualization

Mathematical Question Types - BERTopic

Topics labeled with `openhermes-2.5-mistral-7b`



Dataset curation for retrieval-augmented generation

[A curated dataset of high-school mathematics questions](#)

[Google DeepMind's Mathematics Dataset](#)



Topic-RAG for historical newspapers: Enhancing information retrieval in humanities research through topic-based retrieval-augmented generation

Part of: CHR Expanding the Toolkit: Large Language Models in Humanities Research

Published online by Cambridge University Press: **18 November 2025**

[Keerthana Murugaraj](#) , [Salima Lamsiyah](#), [Marten During](#) and [Martin Theobald](#)

Show author details ▾

Article

Figures

Rapid Responses

Metrics

[Topic-RAG for historical newspapers: Enhancing information retrieval in humanities research through topic-based retrieval-augmented generation | Computational Humanities Research | Cambridge Core](#)

Cohere

🌐 Add languages ▾

Article Talk

Read Edit View history

From Wikipedia, the free encyclopedia

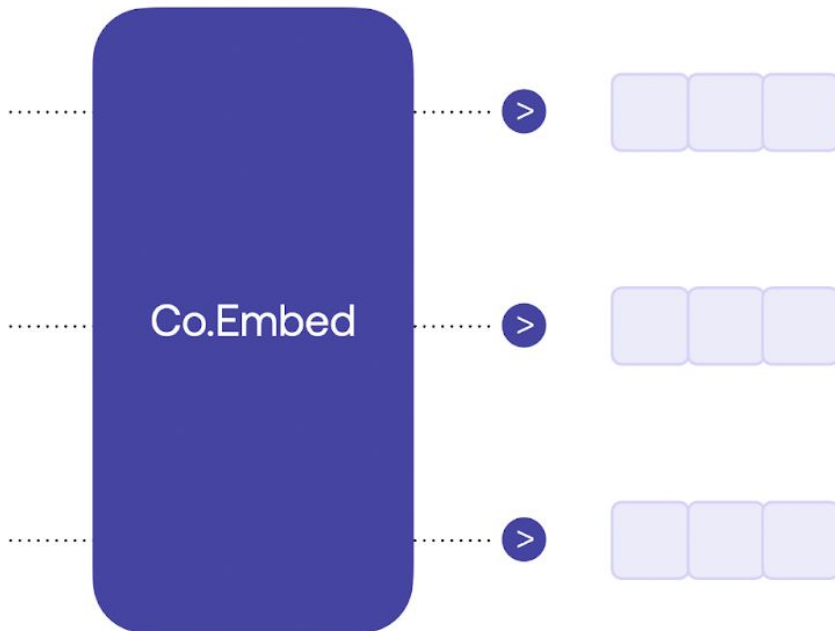
Cohere (stylized as **co:here**) is a Canadian startup that provides [natural language processing](#) models that help companies improve human-machine interactions. Cohere was founded in 2019 by Aidan Gomez, Ivan Zhang, and Nick Frosst.^[1]

History [edit]

In 2017, a team of researchers at Google Brain, which included Aidan Gomez, published a paper called "Attention is All You Need," which introduced the [transformer](#) machine learning architecture, setting state-of-the-art performance on a variety of natural language processing tasks.^{[2][3]} Gomez and Nick Frosst, another researcher at Google Brain, founded Cohere along with Ivan Zhang, with whom Gomez had done research at FOR.ai.^[4] All of the co-founders also attended [University of Toronto](#).^[5] On May 4, 2021, Cohere officially launched.^[6] They also announced that they had been working with a small number of test customers, including Ada Support, a chatbot support service-provider.^{[6][11]} In November 2021, [Google Cloud](#) announced that they would be partnering with Cohere. Google Cloud would help power Cohere's platform using their robust infrastructure, and Cloud's TPUs would be used by Cohere for the development and deployment of their products.^{[7][8]}

Funding [edit]

On September 7th, 2021, Cohere announced that they had raised \$40 million in Series A funding led by [Index Ventures](#); Index Ventures partner [Mike Volpi](#) also joined Cohere's board.^{[9][10]} The round also included Radical Ventures, [Section 32](#), and AI-experts [Geoffrey Hinton](#), [Fei-Fei Li](#), [Pieter Abbeel](#), and [Raquel Urtasun](#).^[11]

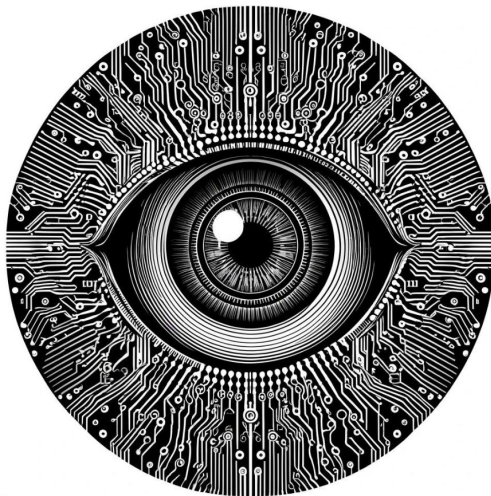


```
from datasets import load_dataset
docs = load_dataset(f"Cohere/wikipedia-22-12-simple-embeddings", split="train")
```

Review Questions

- How do we create text or image embeddings?
- How do we reduce the dimensionality of embeddings?
- What is the purpose of clustering the embeddings?
- What do we tokenize after we have clustered documents?
- Why do we use c-TF-IDF?

Image Embeddings



Learning goals

- Understand dense vector embeddings as learned representations
- Describe how image embeddings are produced in neural networks
- Extract image embeddings out of a pretrained Resnet50
- Inspect image neighborhoods using the cosine similarity metric

Sparse vs dense vector representations

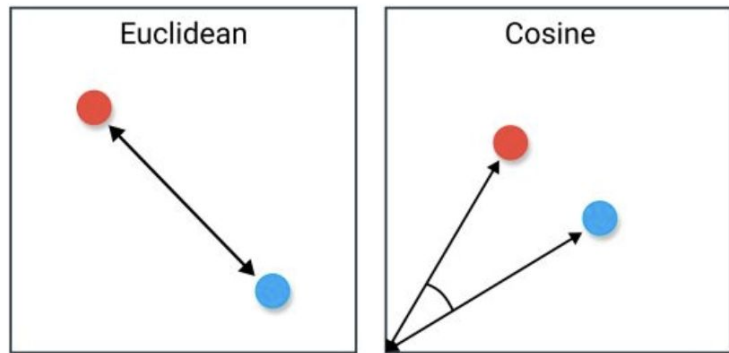
$$\mathbf{x}_1 = \text{[Image of handwritten 3]} \quad \mathbf{x}_2 = \text{[Image of handwritten 3]} \quad 28 \times 28 \text{ pixels} = 784 \text{ values}$$

$$\mathbf{x}_1 \text{ sparse} = \mathbf{x}_2 \text{ sparse} = [0 \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]^\top$$

$$\mathbf{x}_2 \text{ dense} = [0.10 \quad -0.75 \quad 0.35 \quad 0.41 \quad -0.20 \quad 0.89 \quad -0.60 \quad 0.03 \quad 0.59 \quad -0.50]^\top$$

$$\mathbf{x}_1 \text{ dense} = [0.12 \quad -0.85 \quad 0.37 \quad 0.44 \quad -0.22 \quad 0.90 \quad -0.63 \quad 0.01 \quad 0.58 \quad -0.49]^\top$$

Common metrics to compare embeddings

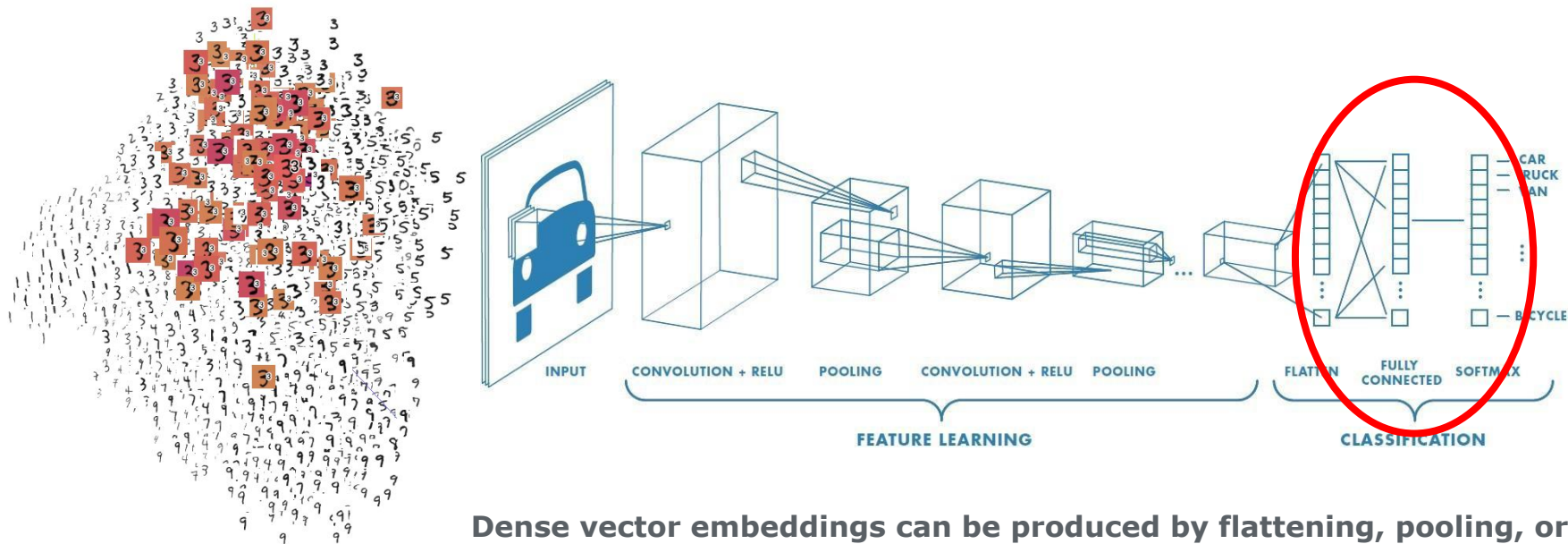


$$d_{\text{euclid}}(x_1, x_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

$$\text{cosine similarity}(x_1, x_2) = \frac{x_1 \cdot x_2}{\|x_1\| \|x_2\|}$$

Embeddings as a byproduct of training

When we train a neural network, we make it **embed** inputs that are semantically similar, close to each other



Dense vector embeddings can be produced by flattening, pooling, or feeding (learned) features to linear layers

[Source of image](#) (try exploring interactively)

Extract embeddings from a pretrained Resnet50

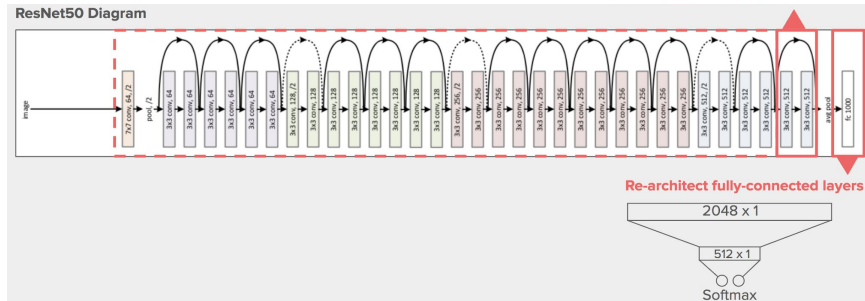
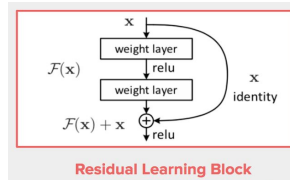
```
import torch
from torchvision import models
```

```
# Load pre-trained ResNet50 weights and transforms
weights = models.ResNet50_Weights.IMAGENET1K_V2
model = models.resnet50(weights=weights)
# Preprocess the image using the transformations from the imagenet dataset
preprocess = weights.transforms()
```

```
# Remove final Imagenet classification layer, keep 2048 dense vector
# Take time to unpack this syntax in the practical notebook
embedder = torch.nn.Sequential(*list(model.children())[:-1])
```

```
# Disable BatchNormalization
embedder.eval()
# Disable gradient computation
with torch.inference_mode():
```

```
    # unsqueeze(0) adds the batch size for inference
    # squeeze() removes the singleton dimensions from the output vector
    embedding = embedder(preprocess(image).unsqueeze(0)).squeeze()
```



Approaches to produce embeddings with neural networks

```
import torch.nn as nn
```

```
# Approach 1: Direct embedding
```

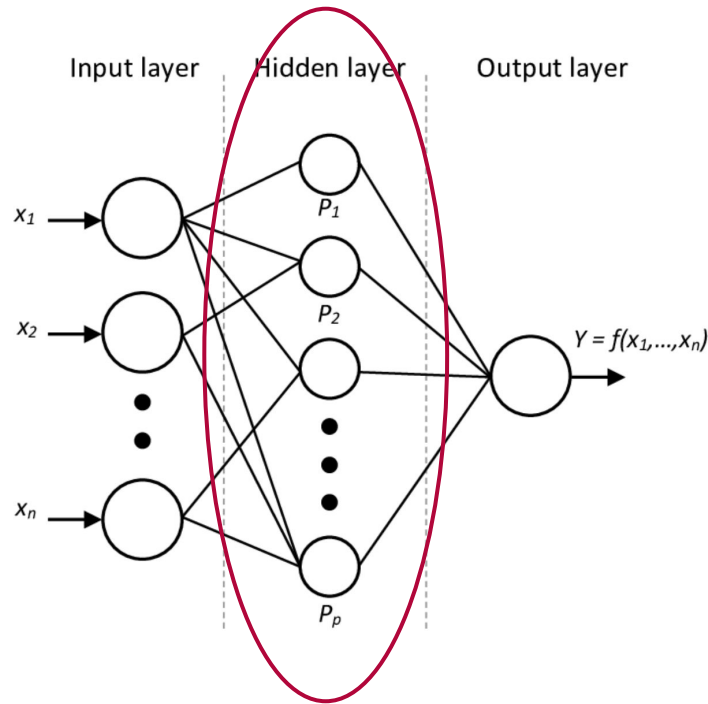
```
linear_embedder = nn.Sequential(  
    nn.Flatten(), # 784  
    nn.Linear(784, 10)  
)
```

```
# Approach 2: Convolution and then fully connected layer
```

```
conv_linear_embedder = nn.Sequential(  
    nn.Conv2d(1, 32, kernel_size=3), # 26x26x32  
    nn.ReLU(),  
    nn.Conv2d(32, 64, kernel_size=3), # 24x24x64  
    nn.ReLU(),  
    nn.Flatten(), # 24*24*64  
    nn.Linear(24*24*64, 10)  
)
```

```
# Approach 3: Global pooling after convolutions
```

```
conv_pool_embedder = nn.Sequential( # MNIST input is size 28x28  
    nn.Conv2d(1, 32, kernel_size=3), # 26x26x32  
    nn.ReLU(),  
    nn.Conv2d(32, 10, kernel_size=3), # 24x24x10  
    nn.ReLU(),  
    nn.AdaptiveAvgPool2d((1, 1)), # 1x1x10  
    nn.Flatten() # 10  
)
```



$$L_{L1} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

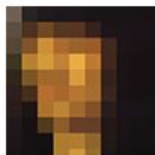
README.md

Art Recommendation system

This is the repository of a portfolio project at [DSR](#). This project aims to identify similar images using pre-trained computer vision networks. For an explanation of the technology see the [technology section](#).

Contributors

- [Catarina Ferreira](#)
- [Gargi Maheshwari](#)

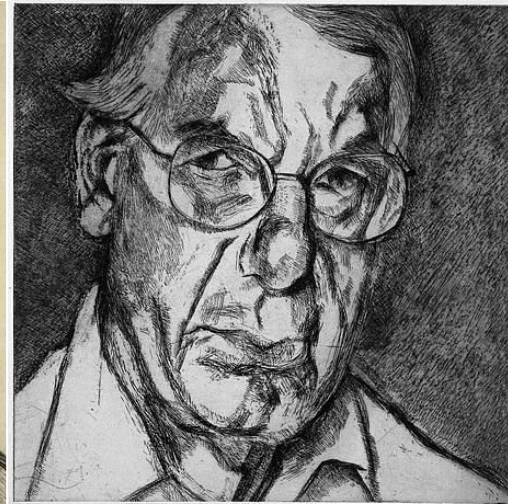


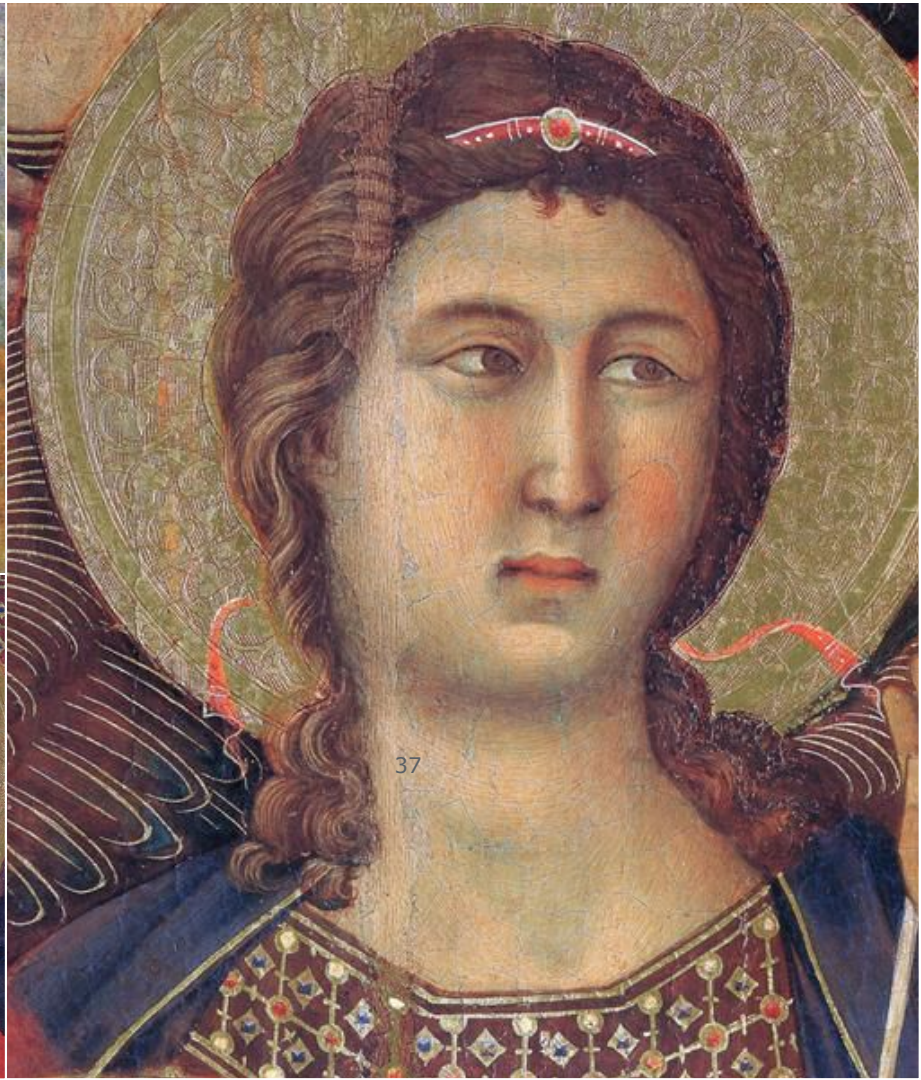
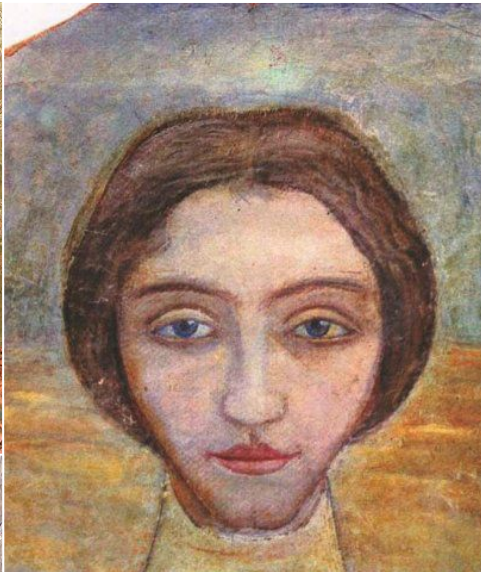
WIKIART
VISUAL ART ENCYCLOPEDIA

<https://github.com/gargimaheshwari/Wikiart-similar-art>





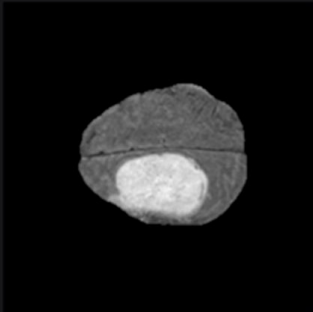




Application: differential diagnosis

PIXEL DIAGNOSE

Select Image



Cancer Type:

- ☒ Gliomas
- ☒ Meningiomas
- ☒ Metastasis

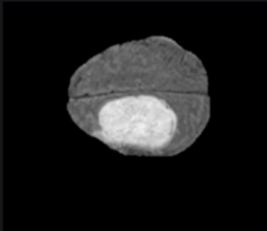
Image Type:

- ☐ t1
- ☐ t1c
- ☐ t2
- ☒ flair
- ☐ seg_t1
- ☐ seg_t1c
- ☐ seg_t2
- ☐ seg_flair

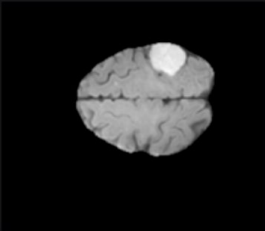
Number of Results per Type:

9

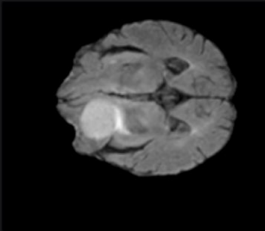
Start Search



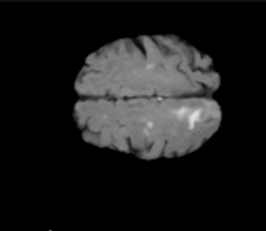
Slice no.: 112 / 150
Similarity Score: 0.9999999



Slice no.: 114 / 150
Similarity Score: 0.8529618



Slice no.: 62 / 150
Similarity Score: 0.82390577



Slice no.: 109 / 150
Similarity Score: 0.8092099



Slice no.: 119 / 150
Similarity Score: 0.8026103

Image from <https://pixel-diagnose.github.io/>

Summary

Embeddings are learned vector representations

- Neural networks learn to map similar images to similar vectors through training
- We measure the similarity of embeddings using metrics like cosine similarity and Euclidean distance

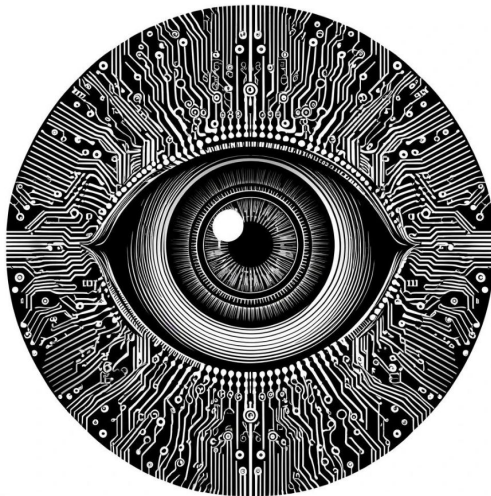
There are different architectural approaches to create embeddings

- We can use linear layers, or flattened and pooled convolutions to produce embeddings
- Imagenet-pretrained models produce rich representations in their embeddings

Embeddings have multiple applications in semantic search and dataset curation

- They can be used for reverse image search, recommendation systems, and deduplication

Contrastive Language-Image Pretraining (CLIP)



Learning goals

- Understand how CLIP models are trained
- Create image and text embeddings with a pretrained CLIP model

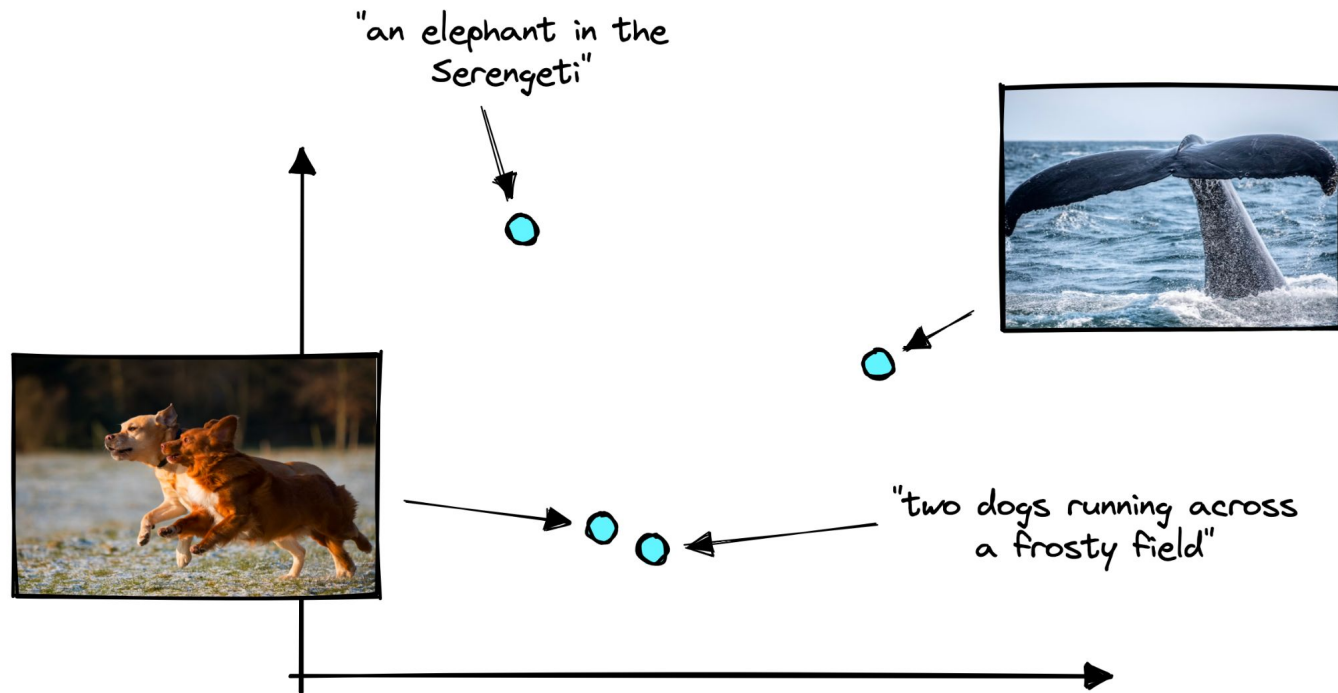
CLIP: ‘Contrastive Language Image Pretraining’

Learning Transferable Visual Models From Natural Language Supervision

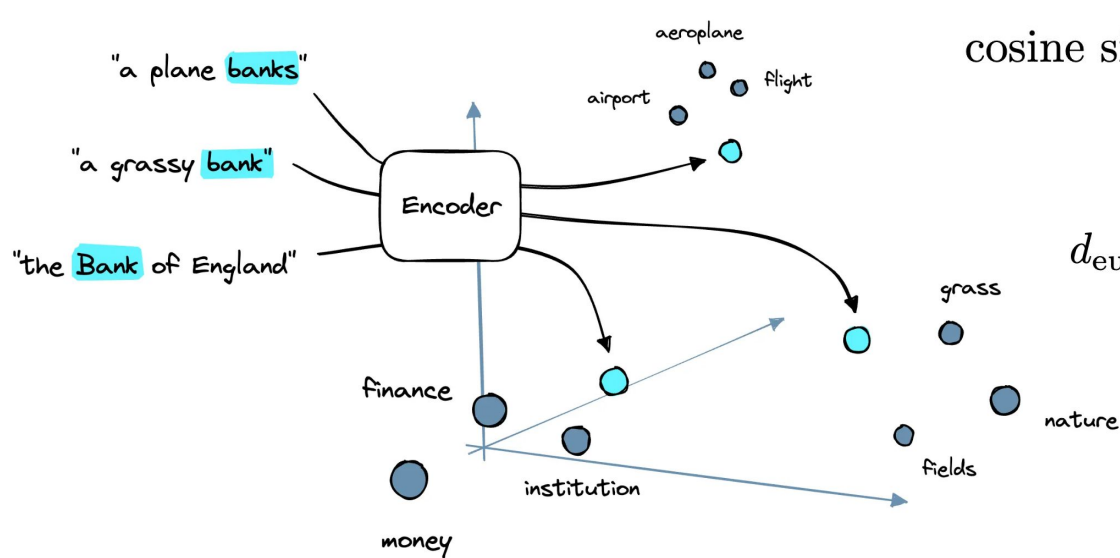
Alec Radford^{*1} Jong Wook Kim^{*1} Chris Hallacy¹ Aditya Ramesh¹ Gabriel Goh¹ Sandhini Agarwal¹
Girish Sastry¹ Amanda Askell¹ Pamela Mishkin¹ Jack Clark¹ Gretchen Krueger¹ Ilya Sutskever¹

- Connects text and images in a shared embedding space
- Created by OpenAI in 2021
- Trained on **400M image-textual description pairs** scraped from the internet
- Predicts the most relevant text snippet given an image, enabling “zero-shot classification”

Aligning text and image embeddings



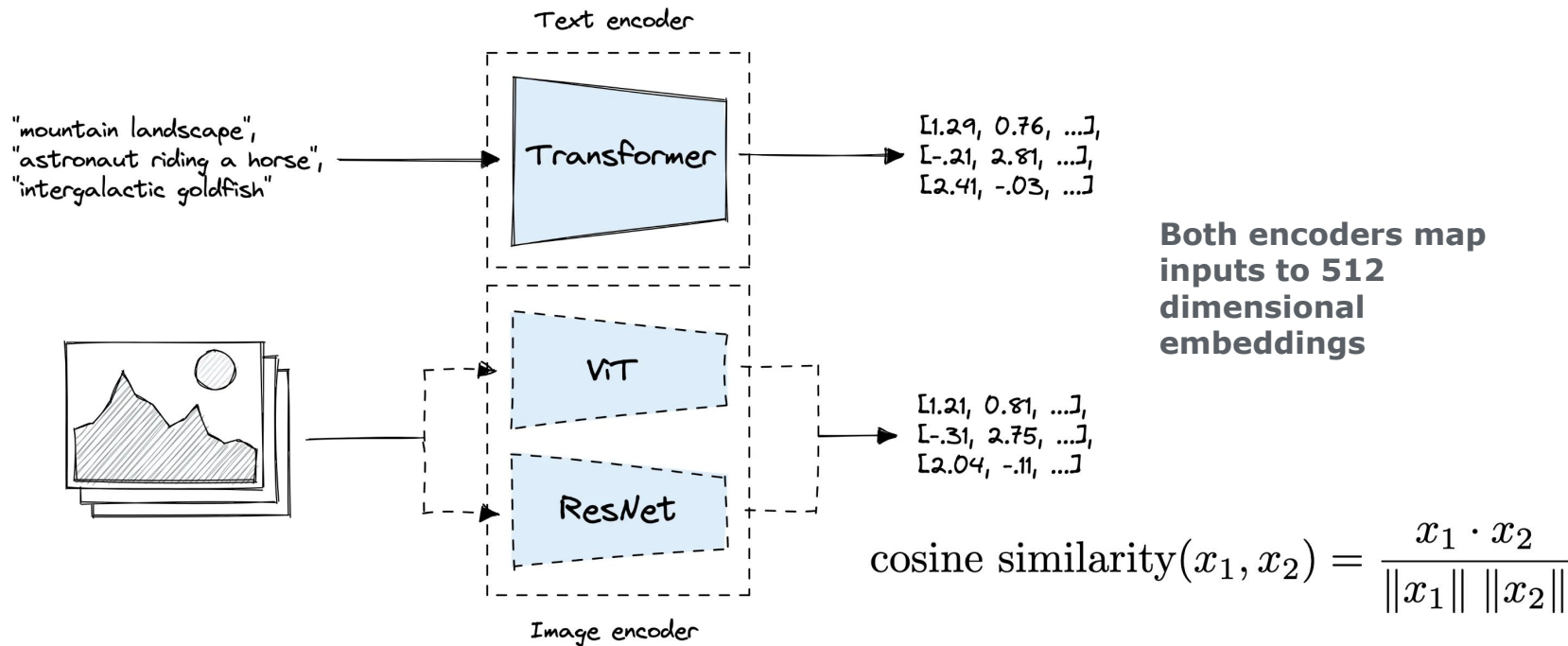
Text encoders



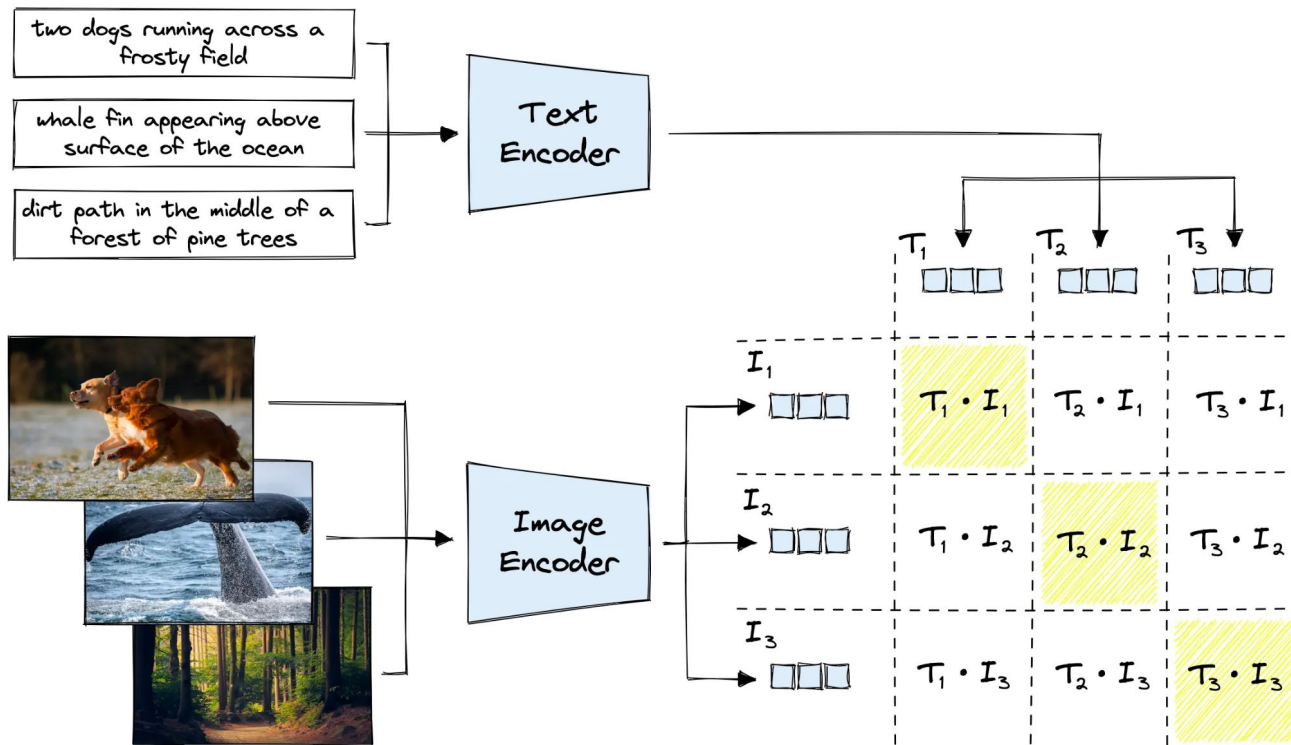
$$\text{cosine similarity}(x_1, x_2) = \frac{x_1 \cdot x_2}{\|x_1\| \|x_2\|}$$

$$d_{\text{euclid}}(x_1, x_2) = \sqrt{\sum_{i=1}^n (x_{1i} - x_{2i})^2}$$

CLIP's architecture



Maximizing cosine similarity of matching text and image embeddings



Training algorithm

```
# image_encoder - ResNet or Vision Transformer
# text_encoder - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l] - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T) #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss = (loss_i + loss_t)/2
```

Figure 3. Numpy-like pseudocode for the core of an implementation of CLIP.

	T ₁	T ₂	T ₃	...	T _N
I ₁	I ₁ ·T ₁	I ₁ ·T ₂	I ₁ ·T ₃	...	I ₁ ·T _N
I ₂	I ₂ ·T ₁	I ₂ ·T ₂	I ₂ ·T ₃	...	I ₂ ·T _N
I ₃	I ₃ ·T ₁	I ₃ ·T ₂	I ₃ ·T ₃	...	I ₃ ·T _N
⋮	⋮	⋮	⋮	⋮	⋮
I _N	I _N ·T ₁	I _N ·T ₂	I _N ·T ₃	...	I _N ·T _N

$$H(p, q) = - \sum_i p(i) \log q(i)$$

$$H(q, p) = - \sum_i q(i) \log p(i)$$

$$\text{symmetric CE loss} = \frac{H(p, q) + H(q, p)}{2}$$

NVIDIA's C-RADIO for Text ↔ Image

Text Prompt: "shoe"



Text Prompt: "helmet"



<https://huggingface.co/nvidia/C-RADIOv4-H>

[C-RADIOv4 \(Tech Report\)](#)

Unsplash Lite 5k

▲ 10

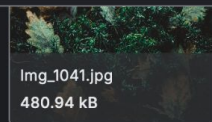
<> Code

Download

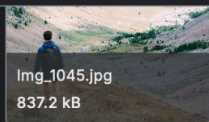


Data Card Code (0) Discussion (0) Suggestions (0)

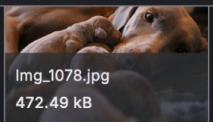
This directory contains images for validating and tuning the performance of a model that is in development.



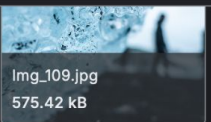
Img_1041.jpg
480.94 kB



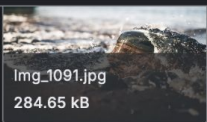
Img_1045.jpg
837.2 kB



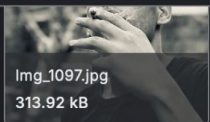
Img_1078.jpg
472.49 kB



Img_109.jpg
575.42 kB



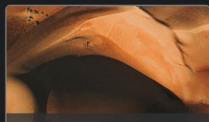
Img_1091.jpg
284.65 kB



Img_1097.jpg
313.92 kB



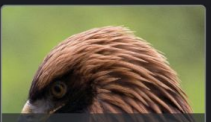
Img_1103.jpg
986.3 kB



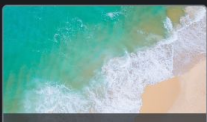
Img_1110.jpg
1.22 MB



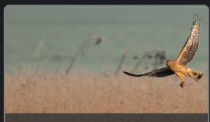
Img_1126.jpg
323.05 kB



Img_113.jpg
375.62 kB



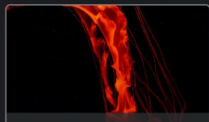
Img_1141.jpg
458.22 kB



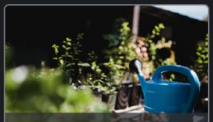
Img_1154.jpg
268.57 kB



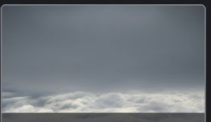
Img_1165.jpg
321.07 kB



Img_1174.jpg
287.13 kB



Img_1179.jpg
198.97 kB



Img_1185.jpg
97.06 kB



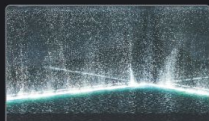
Img_1195.jpg
501.12 kB



Img_1215.jpg
576.55 kB



Img_1242.jpg
1.02 MB



Img_1256.jpg
739.66 kB



Img_1260.jpg
138.31 kB



Img_1261.jpg
577.71 kB



Img_1273.jpg
602.5 kB



Img_1276.jpg
462.49 kB

[Kaggle Unsplash Lite 5k](#)

20 Newsgroups

182

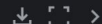
<> Code

Download



Data Card Code (45) Discussion (1) Suggestions (0)

rec.sport.baseball.txt (2.83 MB)



About this file

Suggest Edits

994 messages

Newsgroup: rec.sport.baseball
document_id: 100521
From: admiral@jhunix.hcf.jhu.edu (Steve C Liu)
Subject: spring records

The Orioles' pitching staff again is having a fine exhibition season. Four shutouts, low team ERA, (Well, I haven't gotten any baseball news since March 14 but anyways) Could they contend, yes. Could they win it all? Maybe.

But for all those fans of teams with bad spring records, remember Earl Weaver's first law of baseball (From his book on managing)

No one gives a damn in July if you lost a game in March. :)

BTW, anyone have any idea on the contenders for the O's fifth starter? It's pretty much set that Sutcliffe, Mussina, McDonald and Rhodes are the first four in the rotation.

Here at Johns Hopkins University where the mascot is the Blue Jay :(, their baseball team logo was the Toronto club's logo. Now it's a anatomically correct blue jay. God, can't they think of an original idea? It's even in the same pose as the baltimore oriole on the O's hats. How many people realize that the bird is really called a baltimore oriole?

```
-----
|Admiral Steve C. Liu      Internet Address: admiral@jhunix.hcf.jhu.edu|
|"Committee for the Liberation and Intergration of Terrifying Organisms |
|and their Rehabilitation Into Society" from Red Dwarf - "Polymorph"    |
|****The Bangles are the greatest female rock band that ever existed!****|
|This is a repost of a message I sent to The Open Group mailing list on  |
|the 1st of March 1994. I am not sure if it is still relevant but I am    |
|posting it here for the sake of completeness.                           |
|-----
```

Data Explorer

Version 1 (72.08 MB)

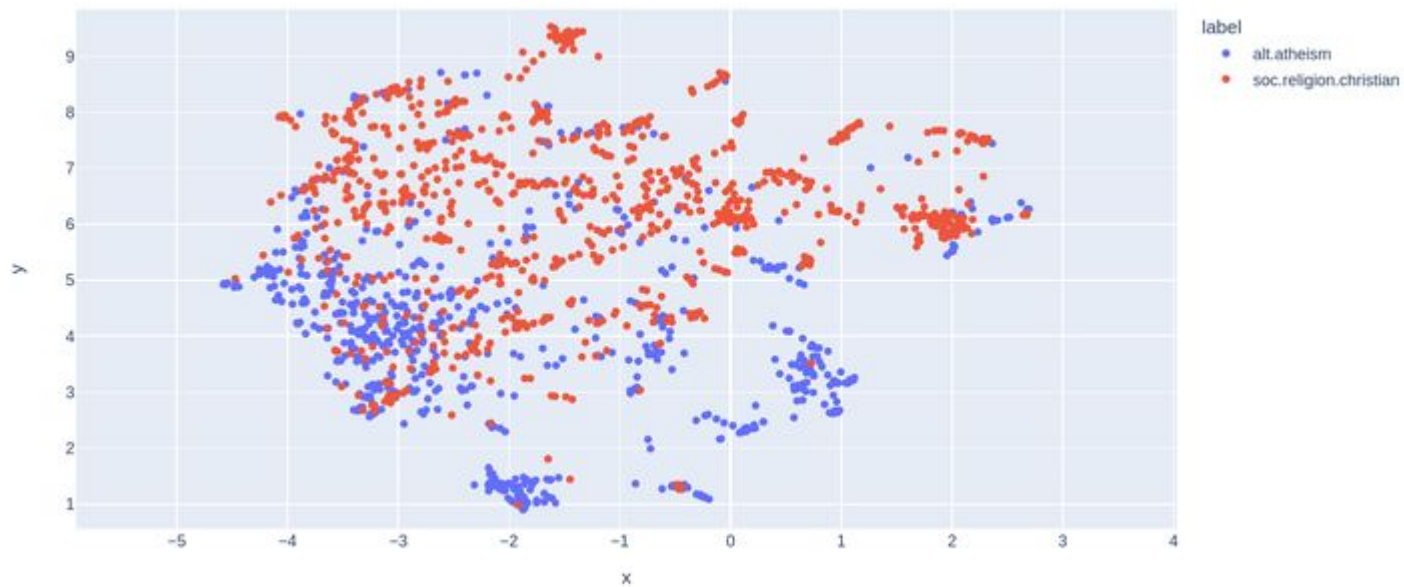
- alt.atheism.txt
- comp.graphics.txt
- comp.os.ms-windows.misc.tx
- comp.sys.ibm.pc.hardware.tx
- comp.sys.mac.hardware.txt
- comp.windows.x.txt
- list.csv
- misc.forsale.txt
- rec.autos.txt
- rec.motorcycles.txt
- rec.sport.baseball.txt**
- rec.sport.hockey.txt
- sci.crypt.txt
- sci.electronics.txt
- sci.med.txt
- sci.space.txt
- soc.religion.christian.txt
- talk.politics.guns.txt
- talk.politics.mideast.txt
- talk.politics.misc.txt
- talk.religion.misc.txt

Summary

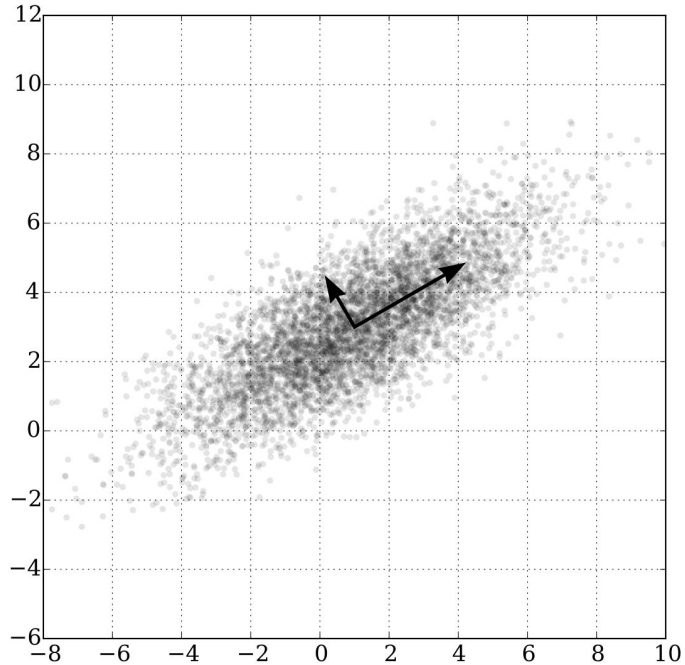
- 21 files
- 2 columns

Kaggle - 20 Newsgroups

Dimensionality Reduction

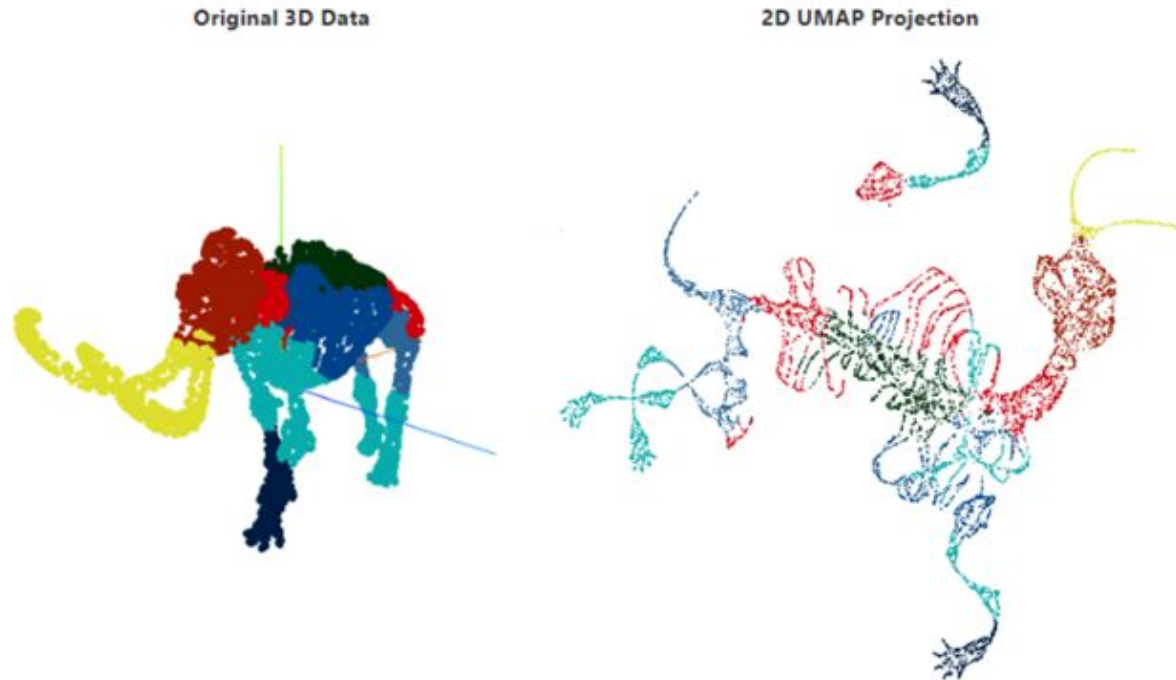


Principal Component Analysis



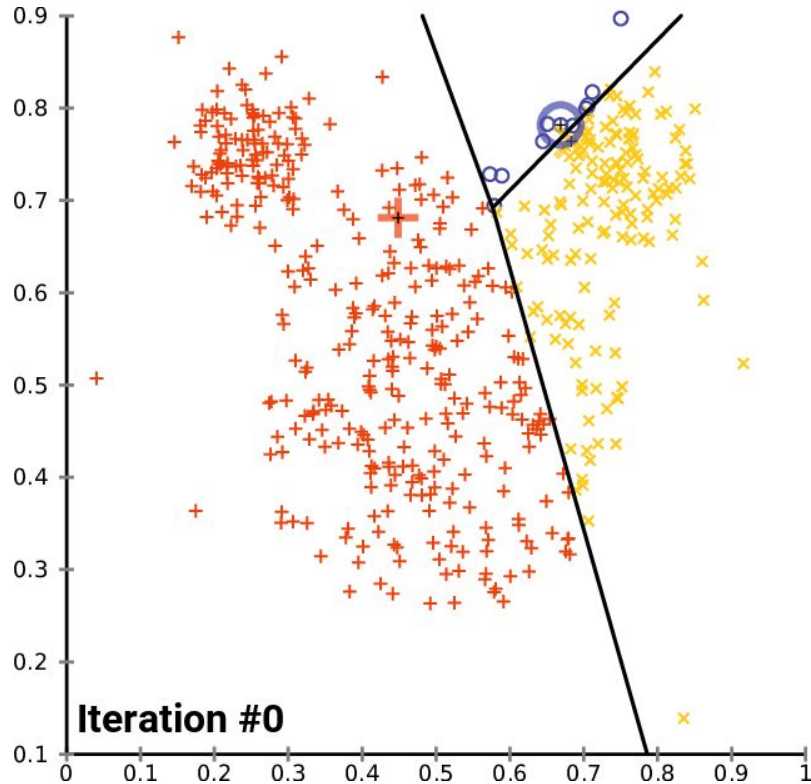
https://en.wikipedia.org/wiki/Principal_component_analysis

UMAP

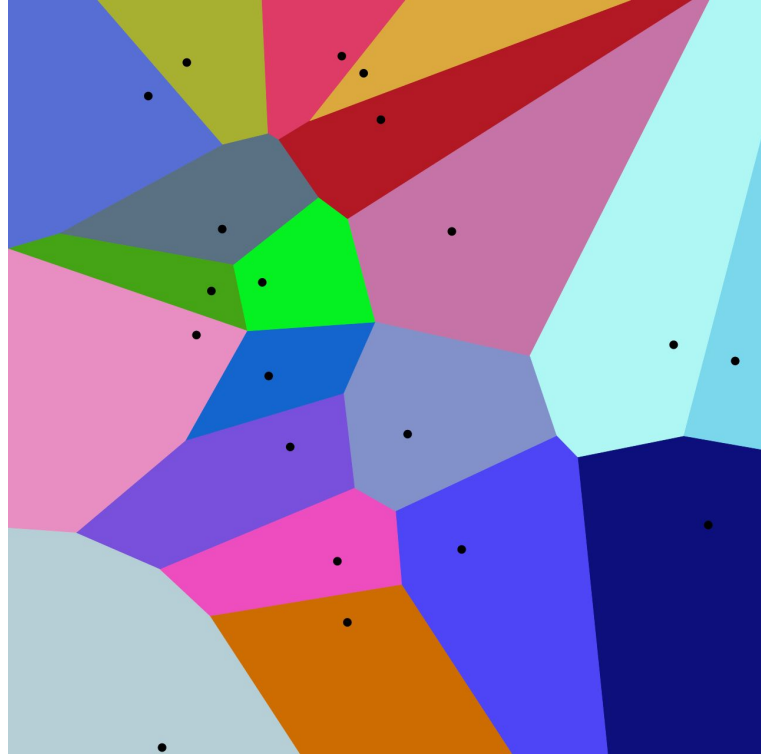


UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction (McInnes et al., 2018)

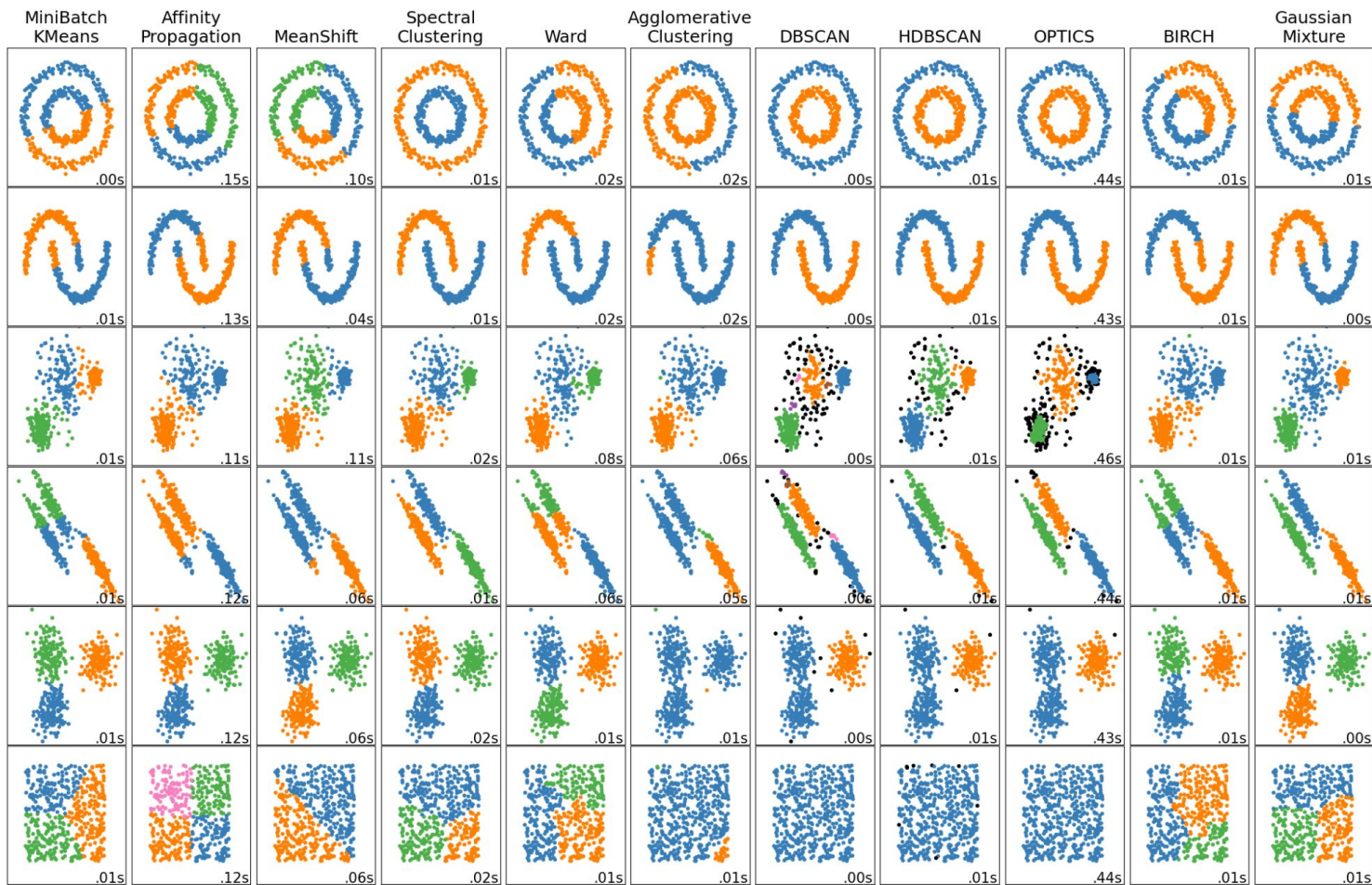
K-means clustering



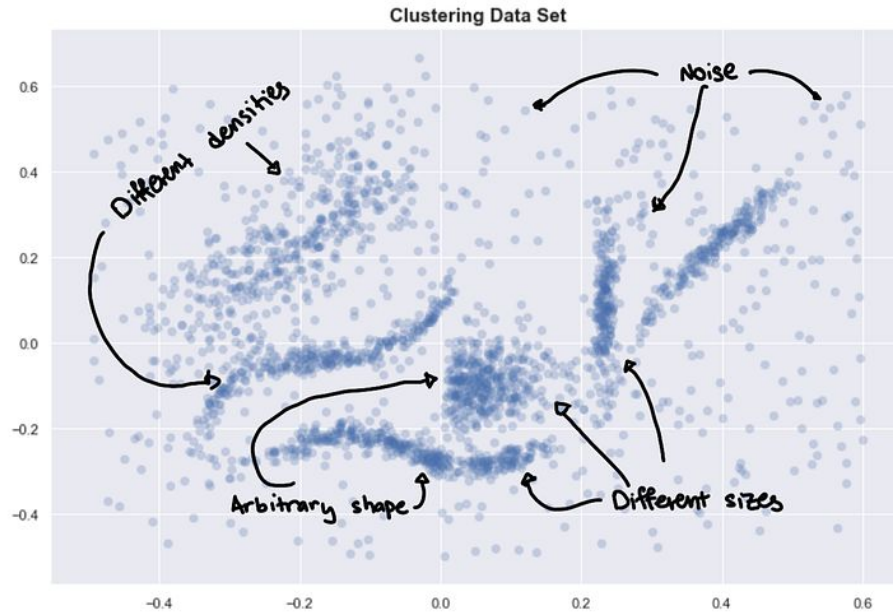
[k-means clustering - Wikipedia](#)



[Voronoi diagram - Wikipedia](#)

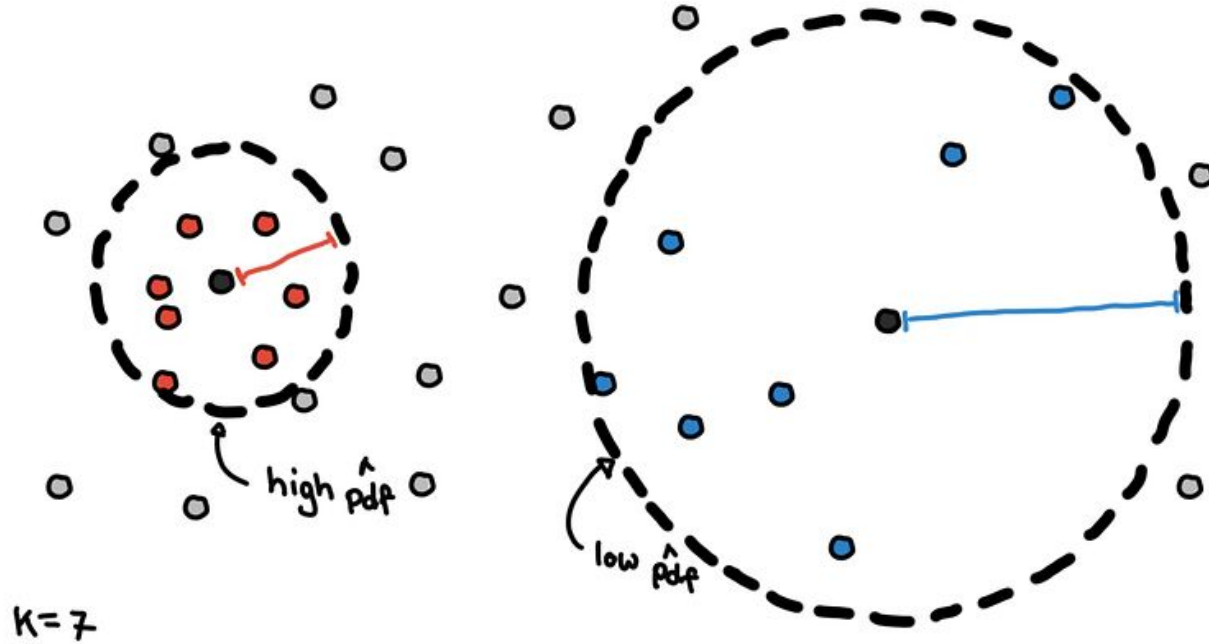


Clustering based on density



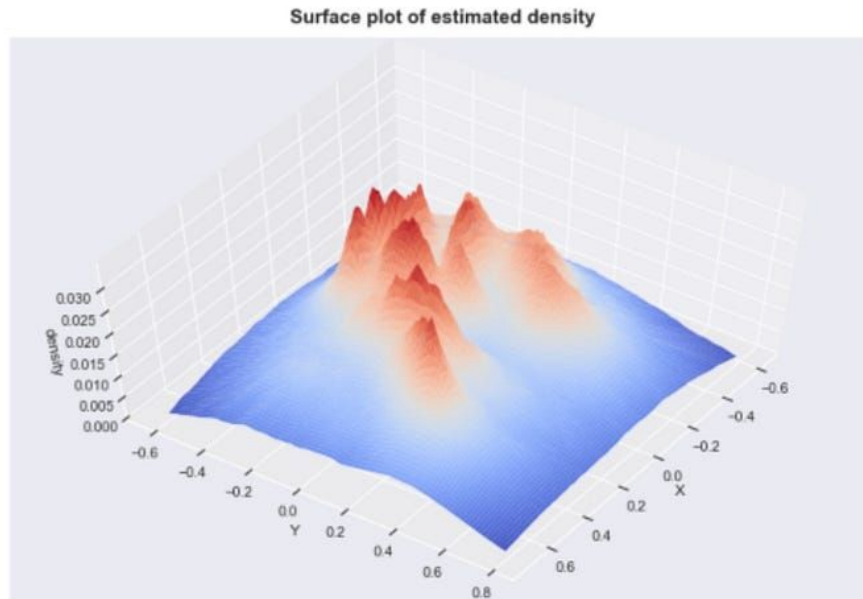
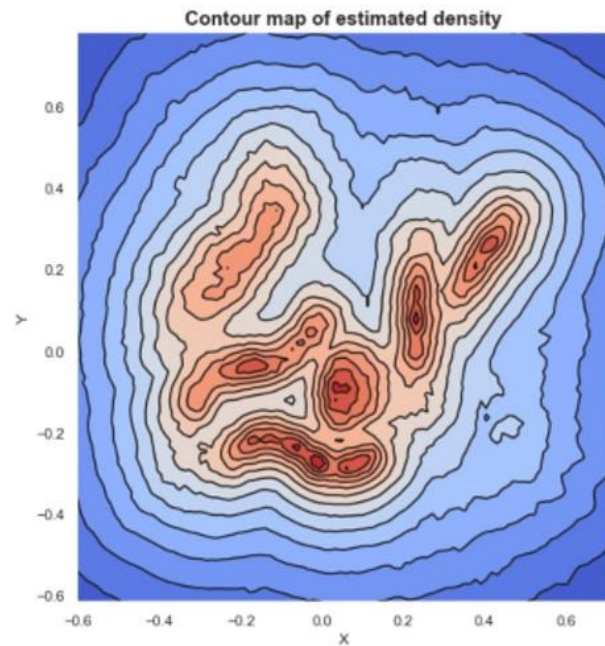
A gentle introduction to HDBSCAN

Clustering with HDBSCAN



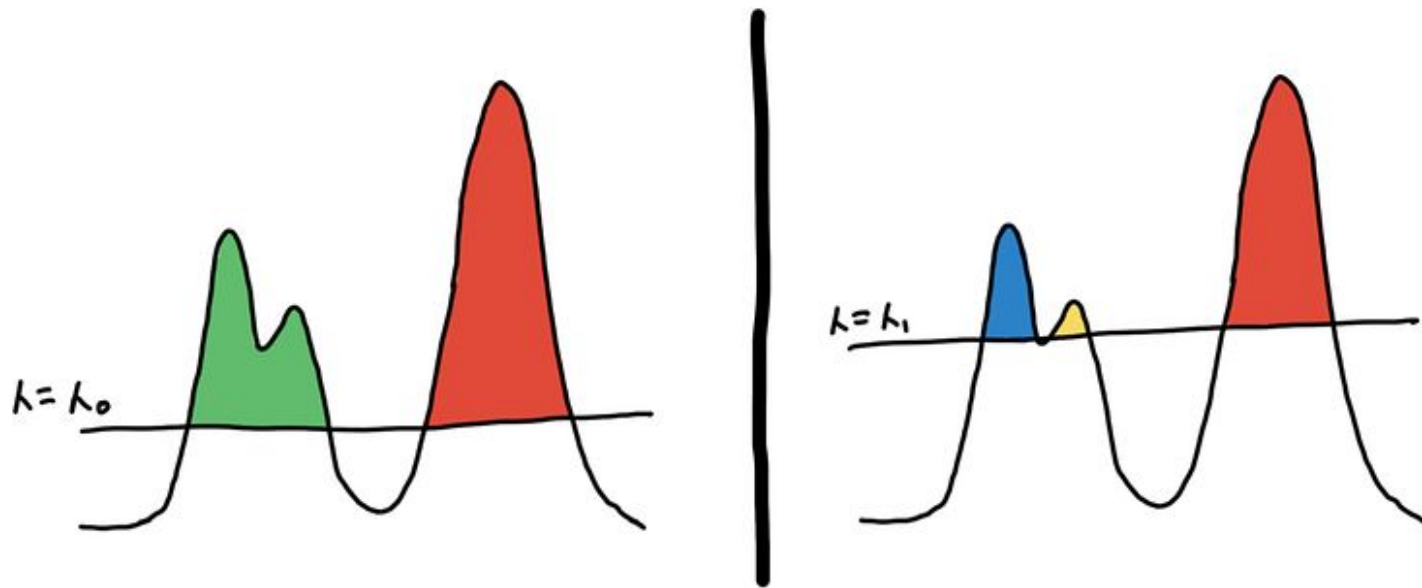
[A gentle introduction to HDBSCAN](#)

Clustering



[A gentle introduction to HDBSCAN](#)

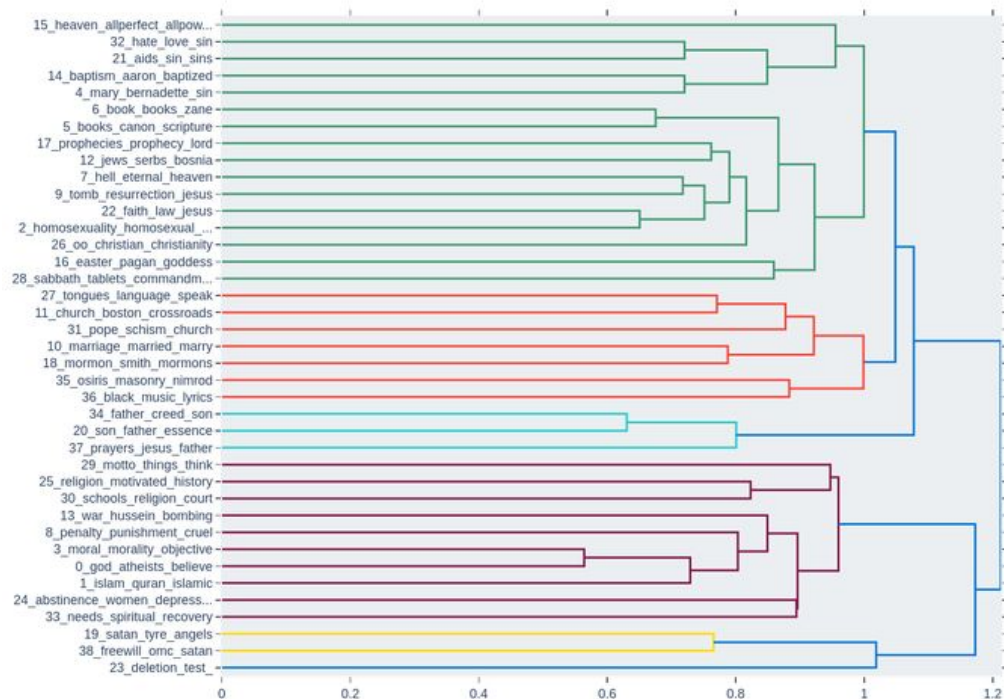
Clustering with HDBSCAN



[A gentle introduction to HDBSCAN](#)

HDBSCAN on 20 Newsgroups dataset

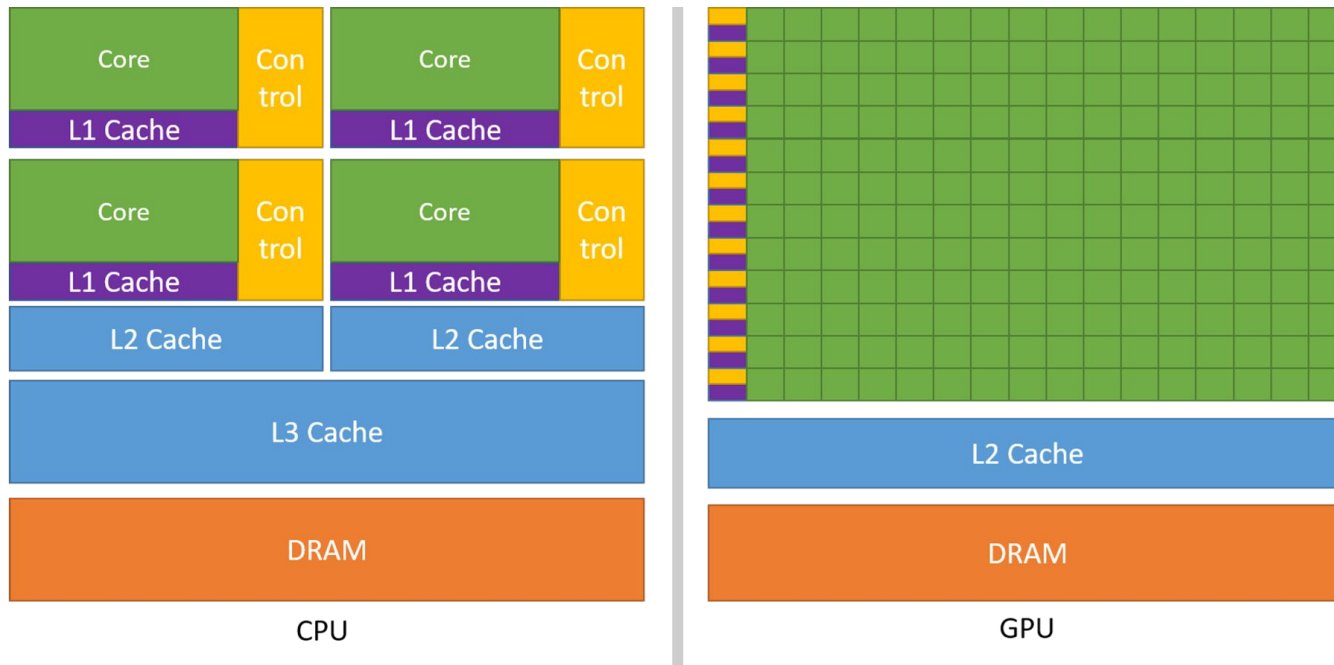
Hierarchical Clustering



● Dataset

CPU and GPU Comparison

GPUs Leverage Parallel Processing to Handle Large Datasets



<https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

BERTopic

Use cuML to Speed Up both UMAP and HDBSCAN through GPU Acceleration

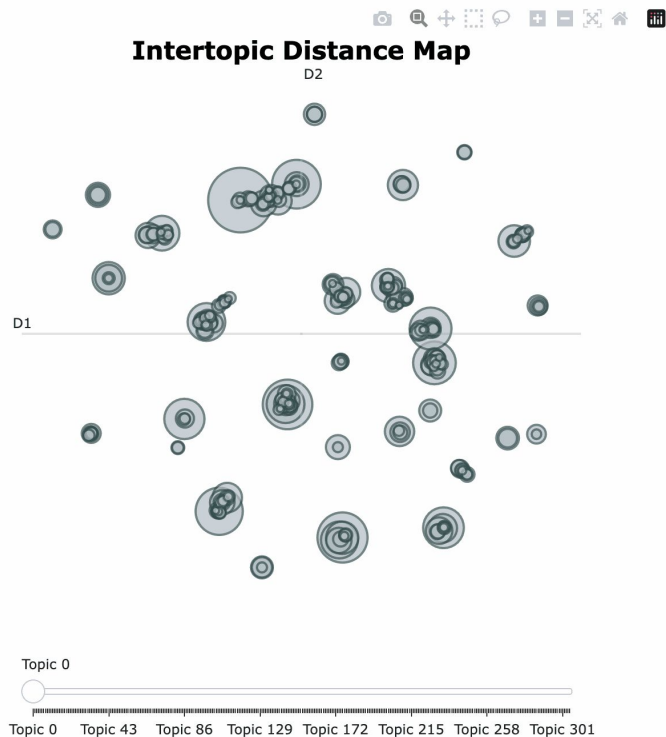
```
from bertopic import BERTopic
from cuml.cluster import HDBSCAN
from cuml.manifold import UMAP
from datasets import load_dataset

# Load IMDb reviews dataset
dataset = load_dataset("imdb", split="train")
docs = dataset["text"] # List of text documents

# Create instances of GPU-accelerated UMAP and HDBSCAN
umap_model = UMAP(n_components=5, n_neighbors=15, min_dist=0.0)
hdbscan_model = HDBSCAN(min_samples=10, gen_min_span_tree=True, prediction_data=True)

# Pass the above models to be used in BERTopic
topic_model = BERTopic(umap_model=umap_model, hdbscan_model=hdbscan_model)
topics, probs = topic_model.fit_transform(docs)

# Visualize the topics
topic_model.visualize_topics()
```



Topic keyword extraction with CountVectorizer and c-TF-IDF

Uses original documents in each cluster

Applies simple tokenization (e.g., word-splitting)

Counts word frequencies within clusters

Selects most frequent/representative words

Uses these words to label topics

Review Questions

- How do we create text or image embeddings?
- How do we reduce the dimensionality of embeddings?
- What is the purpose of clustering the embeddings?
- What do we tokenize after we have clustered documents?
- Why do we use c-TF-IDF?

Resources

- [Topic Modeling | DGX Station at build.nvidia.com](#)
- [Faster Topic Modeling with BERTopic and RAPIDS cuML | by Maarten Grootendorst](#)
- [Accelerate Clustering Algorithms to Achieve the Highest Performance | GTC San Jose 2025 | NVIDIA On-Demand](#)
- [Practical Computer Vision Bootcamp - GitHub Repository](#)